



Healthcare Data Science Research Internship

Predictive Modelling and Evaluation of Chemical Compound Bioactivity Using Machine Learning: A Healthcare Data Science Approach

SUBMITTED BY

PRAVEEN R

BS-MS Student

Indian Institute of Science Education and Research (IISER) Tirupati

SUBMITTED TO

YuvaIntern

Healthcare Data Science Researcher Internship

WEEK 3 SUBMISSION

Predictive Modelling and Evaluation in Healthcare

PROPOSED TOOLS & TECHNOLOGIES

Python | Pandas | NumPy | Scikit-learn | Matplotlib | Seaborn | Logistic
Regression | Random Forest | Support Vector Machine (SVM)

DATE OF SUBMISSION

10/08/2026

Table of Contents

S. No.	Contents	Page No.
1	Abstract	1
2	Introduction	2
3	Healthcare Problem and Project Overview	4
4	Dataset Description and Data Assumption	5
5	Data Preprocessing and Feature Selection	7
6	Predictive Modelling Methodology	8
7	Machine Learning Model Selection	10
8	Model Training, Validation, and Testing	11
9	Hyperparameter Tuning	16
10	Performance Evaluation Metrics	19
11	Results and Critical Evaluation	24
12	Strengths, Limitations, and Ethical Considerations	27
13	Conclusion	29
14	References	30
15	Appendix	31

Abstract

Predictive modelling plays a significant role in healthcare data science by enabling data-driven approaches for disease prediction, drug discovery, and clinical decision support. This report presents the third-week project completed as part of the Healthcare Data Science Research Internship Programme (YUVA), focusing on the development and evaluation of machine learning models to predict the bioactivity of chemical compounds using molecular descriptor data. A synthetic healthcare-related bioactivity dataset was generated to simulate a realistic drug discovery scenario while ensuring ethical compliance and avoiding the use of confidential patient information.

The project involved data preprocessing, feature selection, and the implementation of supervised machine learning algorithms using Python and the Scikit-learn library. The dataset was divided into training and testing subsets, and models including Logistic Regression, Random Forest, and Support Vector Machine (SVM) were developed to classify compounds as biologically active or inactive. Model performance was evaluated using accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC, enabling a comparative assessment of the predictive capabilities of each algorithm.

The study also discusses the strengths, limitations, and ethical considerations of predictive modelling, including data quality, algorithmic bias, model transparency, and the responsible use of artificial intelligence in healthcare. Overall, this project demonstrates a systematic machine learning workflow for healthcare predictive analytics while strengthening practical skills in Python programming, model development, performance evaluation, and scientific documentation. The knowledge and methodologies presented provide a foundation for future applications in computational drug discovery, medicinal chemistry, and healthcare data science.

Introduction

The rapid advancement of digital healthcare technologies has resulted in the generation of vast amounts of biomedical and clinical data, creating unprecedented opportunities for data-driven decision-making in healthcare. Healthcare Data Science integrates statistics, computer science, artificial intelligence, and domain expertise to transform complex healthcare data into meaningful insights that support disease diagnosis, treatment planning, patient outcome prediction, and pharmaceutical research. Among its many applications, predictive modelling has become an essential approach for identifying hidden patterns in data and forecasting future outcomes using machine learning techniques.

Predictive modelling involves the development of computational models that learn relationships between input features and target outcomes from historical or simulated datasets. In healthcare, these models are widely used for disease diagnosis, patient risk assessment, treatment response prediction, hospital readmission analysis, and drug discovery. By leveraging large and complex datasets, predictive models assist researchers and healthcare professionals in making informed decisions, improving diagnostic accuracy, optimizing resource utilization, and accelerating biomedical research.

Machine learning serves as the foundation of modern predictive analytics by enabling computers to learn from data without being explicitly programmed for specific tasks. Supervised learning algorithms, including Logistic Regression, Random Forest, and Support Vector Machine (SVM), are commonly employed for classification problems because of their ability to identify complex relationships among variables and generate reliable predictions. The effectiveness of these models depends on careful data preprocessing, feature selection, model training, validation, and comprehensive performance evaluation.

In computational drug discovery, predictive modelling has become an invaluable tool for identifying biologically active compounds, evaluating molecular properties, and supporting the early stages of pharmaceutical research. Molecular descriptors such as Molecular Weight, LogP, Hydrogen Bond Acceptors (HBA), Hydrogen Bond Donors (HBD), Topological Polar Surface Area (TPSA), and Rotatable Bonds provide quantitative information about the physicochemical characteristics of chemical compounds. Machine learning algorithms can utilize these descriptors to predict compound bioactivity, enabling researchers to prioritize promising drug candidates for further experimental investigation while reducing the time, cost, and resources associated with conventional drug discovery approaches.

This report presents the third-week project completed as part of the Healthcare Data Science Research Internship Programme (YUVA). The study focuses on designing, implementing, and evaluating supervised machine learning models to predict the bioactivity of chemical compounds using a synthetic healthcare-related dataset. The project encompasses data preprocessing, feature selection, model development, training, validation, testing, and comparative performance analysis using widely accepted evaluation metrics, including accuracy, precision, recall, F1-score, confusion matrix, and Receiver Operating Characteristic–Area Under the Curve (ROC-AUC). In addition, the report examines the strengths, limitations, and ethical considerations associated with predictive modelling in healthcare.

Through this project, practical experience was gained in applying Python-based machine learning techniques to healthcare-related predictive analytics while strengthening skills in data preprocessing, model development, performance evaluation, scientific visualization, and technical documentation. The methodologies presented in this report demonstrate the growing importance of predictive modelling in healthcare data science and highlight its potential applications in computational drug discovery, medicinal chemistry, pharmaceutical research, and AI-assisted healthcare research.

1. Healthcare Problem and Project Overview

The rapid advancement of healthcare technologies has generated vast amounts of biomedical and pharmaceutical data, creating significant opportunities for the application of machine learning in predictive analytics. One of the major challenges in modern drug discovery is the identification of biologically active compounds that have the potential to become safe and effective therapeutic agents. Conventional experimental screening methods are often expensive, labour-intensive, and time-consuming, making computational approaches increasingly important for accelerating the drug discovery pipeline and improving research efficiency.

This study addresses the healthcare challenge of predicting the bioactivity of chemical compounds using supervised machine learning techniques. Bioactivity prediction is a critical step in computational drug discovery because it enables researchers to identify compounds that are more likely to interact effectively with biological targets. Reliable prediction of compound bioactivity can reduce the number of compounds requiring experimental validation, thereby lowering research costs and shortening the early stages of pharmaceutical development.

To investigate this problem, a synthetic healthcare-related bioactivity dataset was generated using molecular descriptors, including Molecular Weight, LogP, Hydrogen Bond Acceptors (HBA), Hydrogen Bond Donors (HBD), Topological Polar Surface Area (TPSA), and Rotatable Bonds. These descriptors quantitatively describe the physicochemical properties of chemical compounds and serve as predictive features for developing machine learning classification models.

The study involves the development, training, validation, and evaluation of supervised machine learning models using Python and the Scikit-learn library. Multiple classification algorithms, including Logistic Regression, Random Forest, and Support Vector Machine (SVM), are employed to classify compounds as biologically active or inactive. Model performance is assessed using widely accepted evaluation metrics such as accuracy, precision, recall, F1-score, confusion matrix, and Receiver Operating Characteristic–Area Under the Curve (ROC-AUC), enabling a

comprehensive comparison of the predictive capabilities of each algorithm.

Overall, this study demonstrates the application of machine learning to healthcare-related predictive analytics within the context of computational drug discovery. By integrating data preprocessing, predictive modelling, model validation, and performance evaluation into a systematic workflow, the project illustrates how data-driven approaches can support pharmaceutical research and decision-making. The knowledge and practical experience gained through this work contribute to the development of essential skills in healthcare data science, computational chemistry, medicinal chemistry, and AI-assisted drug discovery.

2. Dataset Description and Data Assumption

The predictive modelling study was conducted using a synthetic healthcare-related bioactivity dataset developed to simulate a realistic computational drug discovery scenario. The use of a synthetic dataset provided a controlled and reproducible environment for machine learning model development while eliminating concerns related to patient privacy, confidential clinical information, and ethical restrictions associated with real-world healthcare datasets. This approach enabled the implementation and evaluation of predictive models without compromising data integrity or research ethics.

The dataset comprises chemical compounds characterized by molecular descriptors that represent their physicochemical properties. These descriptors serve as the independent variables (features) used for predicting compound bioactivity. The selected features include Molecular Weight (MW), LogP, Hydrogen Bond Acceptors (HBA), Hydrogen Bond Donors (HBD), Topological Polar Surface Area (TPSA), and Rotatable Bonds (RB). Each compound is assigned a binary target variable (Bioactivity), where **1** represents a biologically active compound and **0** represents an inactive compound. This binary classification framework is appropriate for supervised machine learning algorithms designed for healthcare prediction tasks.

The synthetic dataset was generated using Python by simulating realistic numerical values within ranges commonly observed for drug-like molecules. The generated data were designed to resemble the physicochemical characteristics of compounds encountered in computational medicinal chemistry studies. Before model development, the dataset underwent preprocessing procedures, including verification of missing values, duplicate record detection, consistency checks, and feature standardization using the StandardScaler algorithm from the Scikit-learn library. These preprocessing steps improved data quality and ensured that numerical features were placed on a comparable scale for efficient model training.

Several assumptions were made during the development of the predictive models. It was assumed that the selected molecular descriptors adequately represent the physicochemical properties influencing compound bioactivity and that the assigned bioactivity labels are suitable for supervised binary classification. The observations were considered independent, and the dataset was assumed to be representative of the underlying data distribution. Furthermore, it was assumed that the relationships between molecular descriptors and bioactivity could be effectively learned by supervised machine learning algorithms, enabling reliable prediction of unseen compounds within the scope of this study.

Overall, the dataset and its underlying assumptions provide a suitable foundation for demonstrating predictive modelling techniques in healthcare data science. The structured representation of molecular descriptors and bioactivity labels supports the development, validation, and comparative evaluation of machine learning models while reinforcing the importance of data quality, preprocessing, and appropriate feature selection in computational drug discovery and healthcare predictive analytics.

3. Data Preprocessing and Feature Selection

Data preprocessing is a fundamental step in the machine learning workflow because the quality of input data directly influences the accuracy, reliability, and generalization capability of predictive models. Before model development, the synthetic healthcare-related bioactivity dataset was systematically examined to identify and address potential data quality issues that could affect the learning process.

The preprocessing workflow began with a comprehensive inspection of the dataset using the Pandas library. The dataset structure, data types, descriptive statistics, and feature distributions were analysed to ensure data consistency and completeness. Missing values and duplicate records were identified using appropriate validation techniques. Since the dataset was generated under controlled conditions, no missing values or duplicate observations were detected. Furthermore, numerical features were examined for potential outliers and inconsistencies to verify that the generated data accurately represented the expected physicochemical characteristics of drug-like molecules.

To improve model performance and ensure fair contribution from all numerical variables, feature scaling was performed using the StandardScaler algorithm from the Scikit-learn library. Standardization transformed each numerical feature to have a mean of zero and a standard deviation of one, eliminating differences caused by varying measurement scales. This preprocessing step is particularly important for algorithms such as Logistic Regression and Support Vector Machine (SVM), whose optimization processes are sensitive to differences in feature magnitude.

Feature selection was carried out by identifying molecular descriptors that are widely recognized as important determinants of compound bioactivity. The selected predictor variables included Molecular Weight (MW), LogP, Hydrogen Bond Acceptors (HBA), Hydrogen Bond Donors (HBD), Topological Polar Surface Area (TPSA), and Rotatable Bonds (RB). These descriptors capture essential physicochemical properties, including molecular size, lipophilicity, hydrogen-bonding capacity, molecular flexibility, and polarity, all of which influence the biological behaviour of

chemical compounds. The Bioactivity column was designated as the target variable and excluded from the predictor feature set during model training.

Following preprocessing and feature selection, the prepared dataset was divided into training and testing subsets using the `train_test_split()` function available in the Scikit-learn library. This approach enabled the models to learn underlying patterns from the training data while evaluating their predictive performance on previously unseen test data, thereby providing an objective assessment of model generalization and minimizing the risk of overfitting.

Overall, the preprocessing and feature selection procedures established a clean, standardized, and well-structured dataset suitable for supervised machine learning. These steps enhanced the reliability of the predictive models and provided a strong foundation for subsequent model training, validation, and performance evaluation in the healthcare bioactivity prediction task.

4. Predictive Modelling Methodology

The predictive modelling methodology adopted in this study follows a structured machine learning workflow designed to develop, train, validate, and evaluate classification models for predicting the bioactivity of chemical compounds. The methodology emphasizes systematic data preparation, appropriate model selection, objective performance evaluation, and reproducibility to ensure reliable and interpretable predictive outcomes.

The workflow commenced with the preparation of a synthetic healthcare-related bioactivity dataset comprising molecular descriptors, including Molecular Weight (MW), LogP, Hydrogen Bond Acceptors (HBA), Hydrogen Bond Donors (HBD), Topological Polar Surface Area (TPSA), and Rotatable Bonds (RB). Following data preprocessing and feature standardization, the dataset was partitioned into training and testing subsets using the `train_test_split()` function available in the Scikit-learn library. This partitioning enabled the models to learn from the training data while evaluating their predictive performance on previously unseen test data,

thereby minimizing overfitting and providing an unbiased estimate of model generalization.

Three supervised machine learning classification algorithms—Logistic Regression, Random Forest, and Support Vector Machine (SVM)—were selected for comparative analysis. Logistic Regression was employed as the baseline classification model because of its simplicity, interpretability, and effectiveness in binary classification tasks. Random Forest was selected as an ensemble learning algorithm capable of capturing complex, non-linear relationships while improving predictive stability through the aggregation of multiple decision trees. Support Vector Machine was included because of its strong performance in high-dimensional feature spaces and its ability to identify optimal decision boundaries for binary classification problems.

Each classification model was trained using the training dataset to learn the relationship between the selected molecular descriptors and the target variable (Bioactivity). Hyperparameter tuning was performed to optimize model performance and improve generalization capability. Following training, the developed models were evaluated using the testing dataset to assess their ability to accurately classify previously unseen compounds.

Model performance was evaluated using widely accepted classification metrics, including accuracy, precision, recall, F1-score, confusion matrix, and the Receiver Operating Characteristic–Area Under the Curve (ROC-AUC). These complementary evaluation measures provide comprehensive insights into classification accuracy, prediction reliability, sensitivity, specificity, and the balance between true positive and false positive predictions. Comparative analysis of these performance metrics enabled the identification of the most suitable algorithm for the selected healthcare bioactivity prediction task.

Overall, the adopted methodology integrates data preprocessing, feature selection, model development, hyperparameter tuning, validation, and performance evaluation into a comprehensive machine learning workflow. This systematic approach provides a robust framework for healthcare predictive analytics and demonstrates the practical application of

supervised machine learning techniques in computational drug discovery, medicinal chemistry, and healthcare data science.

5. Machine Learning Model Selection

The selection of appropriate machine learning algorithms is a fundamental step in developing reliable predictive models for healthcare applications. Different classification algorithms possess unique strengths, assumptions, and learning capabilities that influence their predictive performance. To enable a comprehensive comparison, three widely used supervised machine learning algorithms—Logistic Regression, Random Forest, and Support Vector Machine (SVM)—were selected for this study. These algorithms have been extensively applied in healthcare analytics, bioinformatics, and computational drug discovery because of their effectiveness in solving binary classification problems.

Logistic Regression was selected as the baseline classification model due to its simplicity, computational efficiency, and high interpretability. It estimates the probability that a compound belongs to either the biologically active or inactive class based on the selected molecular descriptors. Although mathematically straightforward, Logistic Regression provides a reliable benchmark against which the performance of more advanced machine learning algorithms can be compared.

Random Forest was chosen because of its capability to model complex and non-linear relationships within the dataset. As an ensemble learning algorithm, it constructs multiple decision trees using randomly selected subsets of the training data and combines their predictions to generate the final classification. This ensemble approach improves predictive accuracy, enhances model robustness, and minimizes the risk of overfitting, making Random Forest particularly suitable for healthcare datasets containing multiple interacting features.

Support Vector Machine (SVM) was included because of its effectiveness in handling high-dimensional datasets and its ability to identify an optimal decision boundary that maximizes the separation between different classes. By constructing a maximum-margin hyperplane, SVM often

achieves excellent classification performance, particularly when the relationships between predictor variables are complex. These characteristics make it a valuable algorithm for bioactivity prediction and computational medicinal chemistry applications.

The selection of these three algorithms enables a comparative assessment of diverse machine learning approaches, ranging from linear statistical models to ensemble learning and margin-based classification techniques. Evaluating their predictive performance provides valuable insights into the strengths and limitations of each algorithm while supporting the identification of the most suitable model for healthcare-related bioactivity prediction.

Overall, the selected machine learning models provide a balanced combination of interpretability, predictive accuracy, computational efficiency, and robustness. Their comparative evaluation establishes a strong methodological framework for this study and demonstrates the practical application of supervised machine learning techniques in healthcare data science, computational drug discovery, medicinal chemistry, and pharmaceutical research.

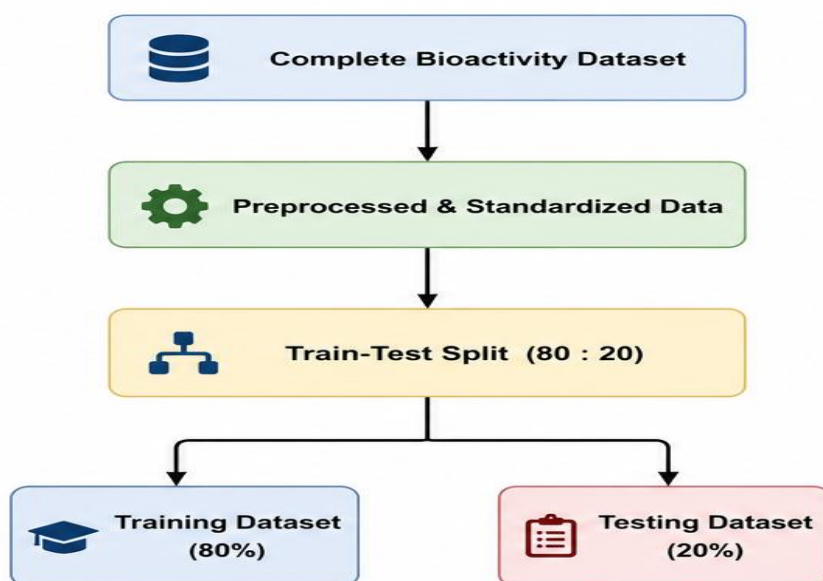
6. Model Training, Validation, and Testing

Machine learning model development involves three fundamental stages: training, validation, and testing. These stages ensure that predictive models learn meaningful relationships from the available data, generalize effectively to previously unseen samples, and generate reliable predictions for real-world healthcare applications. In this study, three supervised machine learning algorithms—Logistic Regression, Random Forest, and Support Vector Machine (SVM)—were implemented using the Scikit-learn library to predict the bioactivity of chemical compounds based on selected molecular descriptors. A systematic workflow was adopted to train, validate, optimize, and evaluate the predictive performance of each model.

6.1 Data Splitting

Before model development, the pre-processed dataset was partitioned into training and testing subsets using the `train_test_split()` function available in the Scikit-learn library. A standard 80:20 split ratio was employed, where 80% of the dataset was allocated for training the machine learning models and the remaining 20% was reserved for independent testing.

The training dataset enabled the algorithms to learn the underlying relationships between the selected molecular descriptors and the target variable (**Bioactivity**). In contrast, the testing dataset remained completely unseen during model development and was used only after training and validation to evaluate predictive performance. This strategy minimizes overfitting, improves model generalization, and provides an unbiased estimate of the model's ability to classify new compounds.



Data Splitting Workflow

Workflow illustrating the division of the pre-processed bioactivity dataset into training (80%) and testing (20%) subsets prior to machine learning model development

6.2 Model Training

The training dataset was used to develop three supervised machine learning classification models:

- Logistic Regression
- Random Forest
- Support Vector Machine (SVM)

Each model was trained using the selected molecular descriptors:

- Molecular Weight (MW)
- LogP
- Hydrogen Bond Acceptors (HBA)
- Hydrogen Bond Donors (HBD)
- Topological Polar Surface Area (TPSA)
- Rotatable Bonds (RB)

These descriptors served as the input features, while the Bioactivity variable represented the target class. During training, each algorithm learned the relationship between the molecular descriptors and the bioactivity status of the compounds by adjusting its internal parameters based on the training data. The objective of this phase was to develop predictive models capable of accurately classifying compounds as biologically active or inactive based on their physicochemical characteristics.

6.3 Model Validation

Following model training, validation was performed to assess the learning capability of each algorithm before final testing. Model validation plays an important role in determining whether a trained model can generalize effectively to new data while minimizing the risks of overfitting and underfitting.

To improve predictive performance, hyperparameter optimization techniques available in the Scikit-learn library were employed during model

development. The validation process ensured that the selected models achieved an appropriate balance between classification accuracy and generalization capability. This stage increased the robustness and reliability of the predictive models before they were evaluated using independent test data.

6.4 Model Testing

After successful training and validation, the optimized models were evaluated using the independent testing dataset. During this phase, each trained model generated predictions for previously unseen samples, and the predicted class labels were compared with the actual bioactivity labels to assess classification performance.

The testing phase provided an objective evaluation of the predictive capability of each algorithm under conditions that closely resemble practical healthcare applications. The results obtained from the testing dataset formed the basis for comparing the three machine learning models.

Model performance was assessed using the following evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Receiver Operating Characteristic (ROC) Curve
- Area Under the ROC Curve (ROC-AUC)

These metrics collectively provided a comprehensive assessment of prediction accuracy, classification reliability, sensitivity, specificity, and overall model performance.

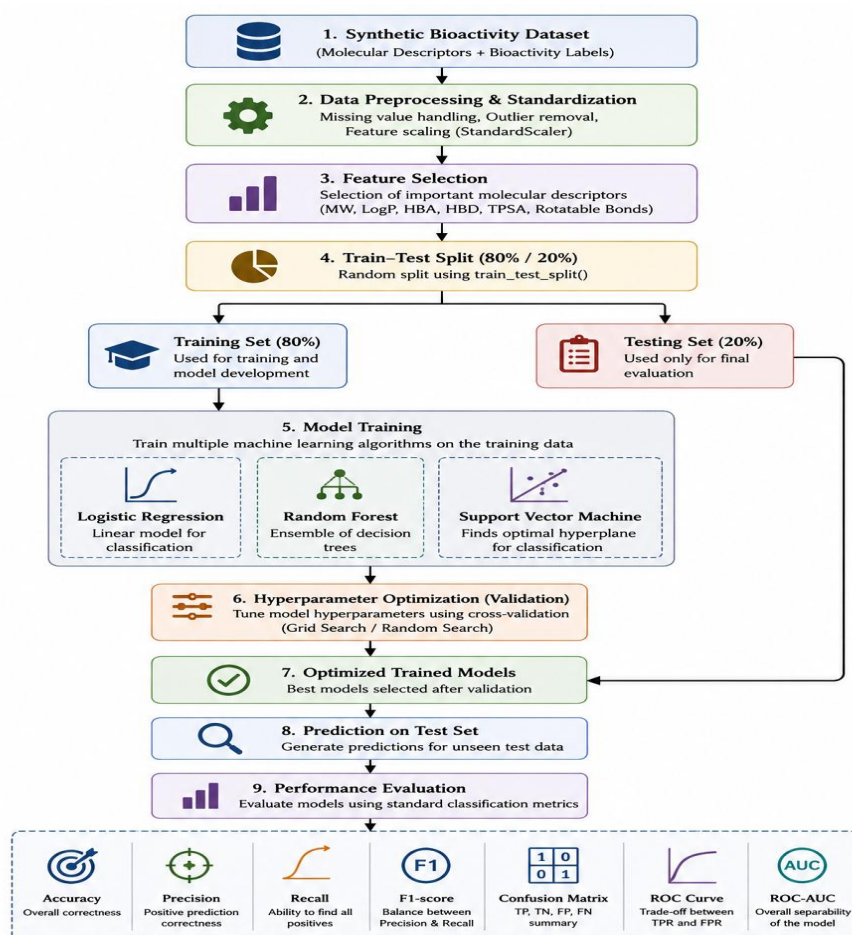
6.5 Machine Learning Workflow

The complete machine learning workflow developed in this study integrates data preprocessing, feature selection, dataset splitting, model training, validation, testing, and performance evaluation into a systematic and reproducible process. The workflow was designed to predict the bioactivity of chemical compounds using selected molecular descriptors and supervised machine learning algorithms.

Initially, the synthetic bioactivity dataset was subjected to data preprocessing and standardization to ensure that the input data were suitable for model development. The relevant molecular descriptors, including Molecular Weight (MW), LogP, Hydrogen Bond Acceptors (HBA), Hydrogen Bond Donors (HBD), Topological Polar Surface Area (TPSA), and Rotatable Bonds (RB), were selected as input features. The dataset was then divided into training and testing subsets using an 80:20 ratio.

The training dataset was used to develop three classification models: Logistic Regression, Random Forest, and Support Vector Machine (SVM). Model validation and hyperparameter optimization were subsequently performed to improve model generalization and identify suitable model configurations. After optimization, the trained models were applied to the independent testing dataset to generate bioactivity predictions for previously unseen compounds.

The final predictions were evaluated using accuracy, precision, recall, F1-score, confusion matrix, ROC curve, and ROC-AUC. The performance obtained from these metrics was used to compare the three algorithms and determine their suitability for the selected bioactivity prediction problem.



Machine Learning Model Development Workflow

Overall machine learning workflow adopted for predictive modelling, illustrating data preprocessing, feature selection, model training, validation, testing, and performance evaluation

7. Hyperparameter Tuning

Hyperparameter tuning is an important step in machine learning model development that involves selecting suitable values for parameters defined before the training process. Appropriate hyperparameter selection can improve model performance, reduce overfitting, and enhance the ability of a model to generalize to previously unseen data. In this study, hyperparameter optimization was considered for the three selected classification algorithms: Logistic Regression, Random Forest, and Support Vector Machine (SVM).

7.1 Purpose of Hyperparameter Tuning

The default parameters provided by machine learning libraries may not always produce optimal predictive performance for a particular dataset. Evaluating different hyperparameter configurations can therefore help identify model settings that provide an appropriate balance between predictive performance and generalization.

For the present bioactivity prediction task, tuning focused on parameters that influence the complexity and learning behaviour of the selected algorithms. The objective was to improve predictive performance while avoiding unnecessary model complexity and overfitting.

7.2 Hyperparameters Considered

Different hyperparameters influence the behaviour of each machine learning algorithm. For Logistic Regression, the regularization parameter controls the complexity of the model. For Random Forest, parameters such as the number of decision trees and maximum tree depth influence model complexity and predictive performance. For SVM, the regularization parameter and kernel configuration influence the decision boundary and classification behaviour.

The principal hyperparameters considered are summarized below.

Model	Hyperparameter	Purpose
Logistic Regression	C	Controls the strength of regularization
Random Forest	n_estimators	Determines the number of decision trees
Random Forest	max_depth	Controls the maximum depth of individual trees
SVM	C	Controls the trade-off between margin and classification errors
SVM	kernel	Determines the type of decision boundary

7.3 Hyperparameter Optimization

The selected hyperparameters were evaluated during model development to identify configurations capable of providing reliable predictive performance. The validation procedure was used to compare the different configurations and determine suitable parameter settings for each algorithm.

The optimization process focused on improving model generalization rather than simply maximizing performance on the training dataset. This approach helps reduce the possibility of overfitting and provides a more reliable basis for evaluating the predictive capabilities of the developed models.

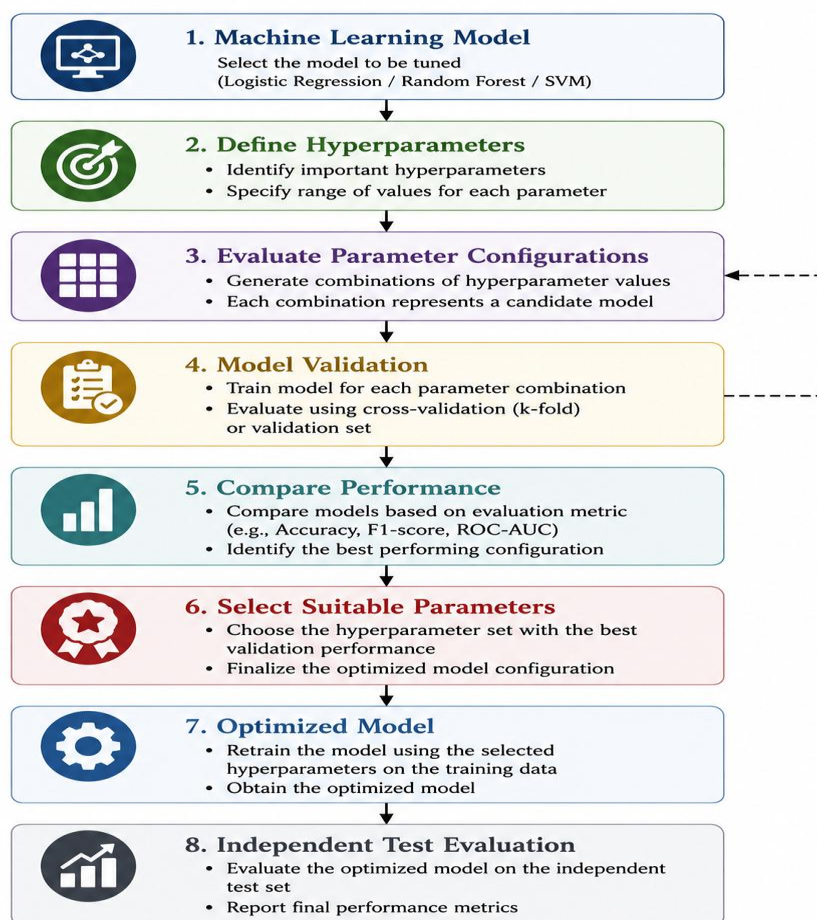
7.4 Selection of the Optimized Model

Following hyperparameter optimization, the resulting model configurations were compared using appropriate classification performance measures. The configuration demonstrating the most suitable validation performance was selected for subsequent evaluation using the independent testing dataset.

The final performance comparison of the optimized Logistic Regression, Random Forest, and SVM models is presented in the Results and Discussion section. This comparison provides the basis for identifying the most suitable algorithm for the selected chemical bioactivity prediction task.

Model	Best Hyperparameters	Best CV F1-Score
Logistic Regression	$C = 1$, solver = liblinear	0.7422
Random Forest	$n_estimators = 100$, $max_depth = 10$, $min_samples_split = 5$	0.7323
Support Vector Machine (SVM)	$C = 10$, kernel = linear, gamma = scale	0.7294

Optimized Hyperparameters and Cross-Validation Performance of the Selected Machine Learning Models



Hyperparameter Tuning Workflow

Overall, hyperparameter tuning provides an important optimization stage between initial model development and final performance evaluation. It helps ensure that the selected machine learning algorithms are appropriately configured for the bioactivity prediction problem and can generalize effectively to previously unseen chemical compounds.

8. Performance Evaluation Metrics

Evaluating the performance of a machine learning model is essential for determining its predictive capability and reliability. In this study, the performance of Logistic Regression, Random Forest, and Support Vector Machine (SVM) models was assessed using multiple classification metrics. Since the objective was to classify chemical compounds as biologically active or inactive, both overall classification performance and the model's ability to correctly identify active compounds were considered.

The evaluation was performed using the independent testing dataset, which was not used during model training. The selected metrics included Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC. Using multiple metrics provides a more comprehensive assessment than relying on accuracy alone.

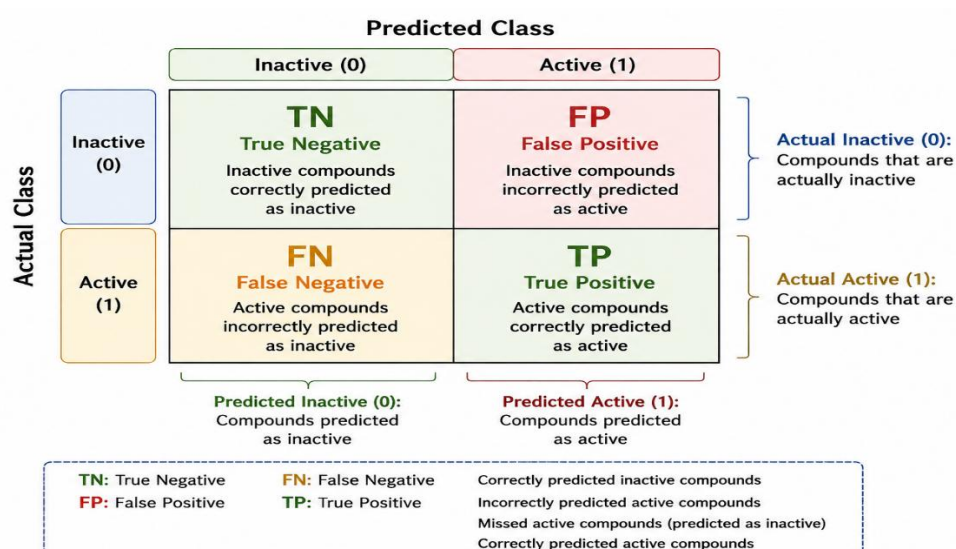
8.1 Confusion Matrix

A confusion matrix provides a detailed representation of the classification results by comparing the predicted classes with the actual classes. For the present study, the two classes were **Active** and **Inactive**.

The confusion matrix consists of four components:

- **True Positive (TP):** Active compounds correctly classified as active.
- **True Negative (TN):** Inactive compounds correctly classified as inactive.
- **False Positive (FP):** Inactive compounds incorrectly classified as active.
- **False Negative (FN):** Active compounds incorrectly classified as inactive.

The confusion matrix was used to identify the types of classification errors made by each machine learning model.



Confusion Matrix Representation

8.2 Accuracy

Accuracy represents the proportion of correctly classified samples out of the total number of samples. It provides an overall measure of classification performance.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

A higher accuracy indicates that the model correctly classified a larger proportion of the compounds. However, accuracy alone may not provide a complete picture when the classes are imbalanced.

8.3 Precision

Precision measures the proportion of compounds predicted as active that were actually active.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

A high precision value indicates that the model produces relatively few false-positive predictions. In the context of bioactivity prediction, high precision means that compounds predicted to be biologically active are more likely to actually belong to the active class.

8.4 Recall

Recall, also known as Sensitivity, measures the proportion of truly active compounds that were correctly identified by the model.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Recall is particularly important in bioactivity prediction because failing to identify an active compound may result in a potentially useful candidate being overlooked. Therefore, a higher recall indicates a greater ability of the model to detect active compounds.

8.5 F1-Score

The F1-score provides a balance between precision and recall. It is particularly useful when both false-positive and false-negative predictions need to be considered.

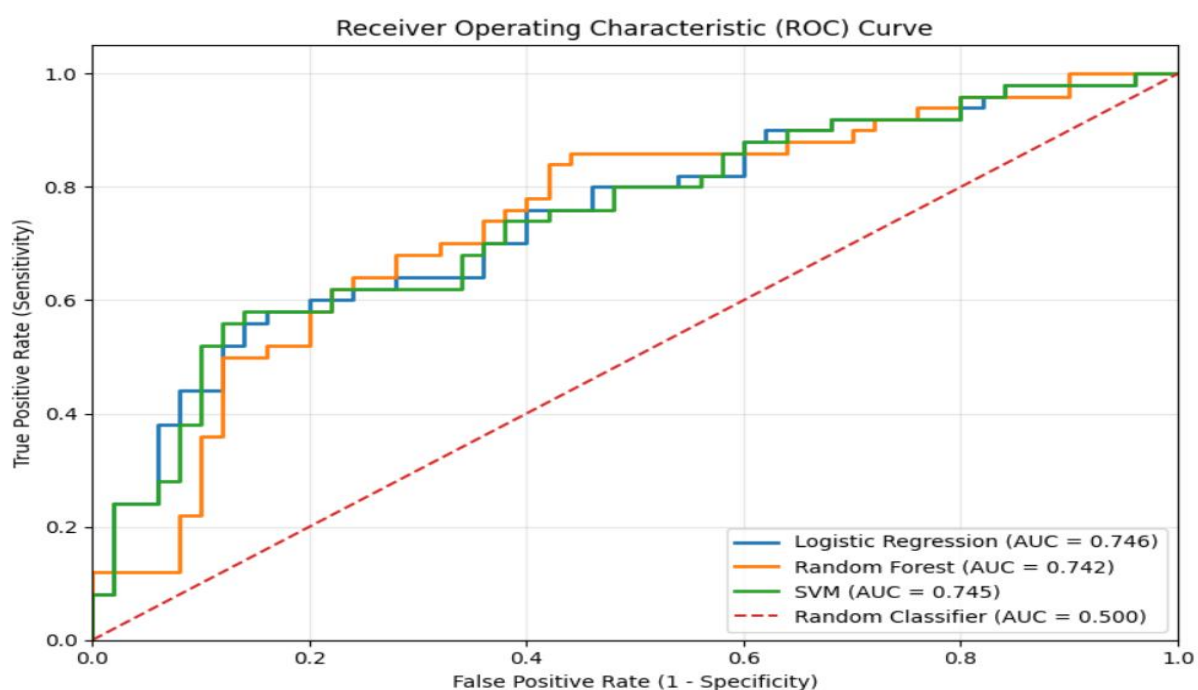
$$F1\text{-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A higher F1-score indicates that the model achieves a better balance between correctly identifying active compounds and minimizing incorrect active predictions.

8.6 Receiver Operating Characteristic (ROC) Curve

The Receiver Operating Characteristic (ROC) curve is a graphical method used to evaluate the classification performance of a model across different classification thresholds. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR).

The ROC curve helps assess how effectively a model distinguishes between active and inactive compounds at different decision thresholds.



ROC Curve

The ROC curves demonstrate the ability of the three machine learning models to distinguish between biologically active and inactive compounds. Logistic Regression achieved the highest ROC-AUC value of 0.746, followed closely by SVM with an AUC of 0.745 and Random Forest with an AUC of 0.742. The ROC-AUC values were above the 0.500 random-classifier reference; however, because the bioactivity labels were synthetically generated and not derived from experimentally measured biological activity, these results should not be interpreted as evidence of genuine biological predictive capability. However, the relatively close AUC values suggest that none of the three algorithms demonstrates a substantial advantage over the others for this synthetic dataset. Logistic Regression showed the highest AUC, although the difference from SVM and Random Forest was small. These results indicate moderate predictive discrimination and suggest that the selected molecular descriptors contain useful information for distinguishing active and inactive compounds, while also indicating scope for further improvement through additional features, larger datasets, and more advanced modelling approaches.

8.7 ROC-AUC

The Area Under the Receiver Operating Characteristic Curve (ROC-AUC) provides a single numerical measure of the model's ability to distinguish between the two classes.

The AUC value generally ranges from 0 to 1, where a value closer to 1 indicates stronger discriminatory performance. A model with an AUC close to 0.5 has performance approximately equivalent to random classification.

ROC-AUC was therefore used alongside accuracy, precision, recall, and F1-score to provide a more comprehensive evaluation of the classification models.

8.8 Comparative Evaluation

The performance metrics obtained from the testing dataset were used to compare Logistic Regression, Random Forest, and SVM. The combination of accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC provided information about both the overall classification performance and the ability of each model to identify biologically active compounds.

The numerical results and graphical evaluation of the three models are presented in the following Results and Critical Evaluation section. This comparison allows the most suitable model for the selected bioactivity prediction task to be identified based on multiple performance criteria rather than a single metric.

Overall, the selected evaluation metrics provide a systematic framework for assessing the reliability, classification capability, and generalization performance of the developed machine learning models in the context of healthcare-related chemical bioactivity prediction.

9. Results and Critical Evaluation

The developed machine learning models were evaluated using an independent testing dataset containing 100 samples, while 400 samples were used for training. The optimized Logistic Regression, Random Forest, and Support Vector Machine (SVM) models were evaluated using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix analysis. The results obtained from the testing dataset are summarized below.

9.1 Model Performance Comparison

MODEL PERFORMANCE COMPARISON						
	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.64	0.6400	0.64	0.6400	0.7456
1	Random Forest	0.68	0.6731	0.70	0.6863	0.7424
2	SVM	0.65	0.6596	0.62	0.6392	0.7448

The results show that Random Forest achieved the highest accuracy (0.680), precision (0.6731), recall (0.700), and F1-score (0.6863) among the three models. Therefore, based on the F1-score used in the model selection process, Random Forest was identified as the best-performing model.

Logistic Regression achieved the highest ROC-AUC of 0.7456, followed closely by SVM with 0.7448 and Random Forest with 0.7424. The small differences between the ROC-AUC values indicate that the three models have relatively similar discriminatory capabilities.

9.2 Confusion Matrix Analysis

The confusion matrices provide further information about correct and incorrect predictions made by the models.

For Logistic Regression, the confusion matrix was:

```
[[32 18]
```

```
[18 32]]
```

The model correctly classified 32 inactive and 32 active compounds. It incorrectly classified 18 inactive compounds as active and 18 active compounds as inactive.

For Random Forest, the confusion matrix was:

```
[[33 17]
```

```
[15 35]]
```

Random Forest correctly classified 33 inactive and 35 active compounds. It produced 17 false-positive and 15 false-negative predictions. This represents the strongest balance among the three models in terms of correctly identifying both classes.

For SVM, the confusion matrix was:

```
[[34 16]
```

```
[19 31]]
```

SVM correctly classified 34 inactive and 31 active compounds, with 16 false-positive and 19 false-negative predictions.

Overall, Random Forest correctly identified the largest number of active compounds and produced the lowest number of false-negative predictions among the three models.

9.3 ROC-AUC Analysis

The ROC-AUC values obtained from the testing dataset were 0.7456 for Logistic Regression, 0.7424 for Random Forest, and 0.7448 for SVM. All three values are above the random-classifier reference value of 0.500,

indicating that the models learned useful patterns for distinguishing active and inactive compounds.

Although Logistic Regression produced the highest ROC-AUC, the difference between the three models was very small. Therefore, ROC-AUC alone does not provide strong evidence that Logistic Regression substantially outperformed Random Forest or SVM.

9.4 Critical Evaluation

The models demonstrated moderate predictive performance, with testing accuracy ranging from 0.640 to 0.680. Random Forest achieved the highest F1-score (0.6863) and recall (0.700), while Logistic Regression achieved the highest ROC-AUC (0.7456).

The moderate performance may be associated with the limited number of molecular descriptors used in the study. Molecular Weight, LogP, HBA, HBD, TPSA, and Rotatable Bonds provide useful physicochemical information but do not fully capture the structural and biological factors influencing compound bioactivity.

Additionally, the dataset was synthetically generated and therefore may not reproduce the complex relationships present in experimentally measured bioactivity data. The results should consequently be interpreted as a demonstration of the machine learning workflow rather than experimentally validated predictions.

9.5 Overall Assessment

Based on the **F1-score**, Random Forest was selected as the best-performing model, achieving an accuracy of 68%, recall of 0.700, and F1-score of 0.6863. However, Logistic Regression produced the highest ROC-AUC of 0.7456, demonstrating that model performance depends on the evaluation metric considered.

Overall, the results demonstrate the potential of machine learning for preliminary bioactivity classification while highlighting the importance of dataset quality, feature selection, and appropriate model evaluation. Further improvement could be achieved using larger experimentally validated datasets and additional molecular descriptors.

10. Strengths, Limitations, and Ethical Considerations

The predictive modelling approach used in this project provides a practical demonstration of applying machine learning to chemical bioactivity prediction. However, like any computational study, the approach has several strengths and limitations that should be considered when interpreting the results.

10.1 Strengths

The major strengths of the project include:

- **Systematic workflow:** The study follows a structured process involving preprocessing, feature selection, model training, hyperparameter tuning, and performance evaluation.
- **Multiple machine learning models:** Logistic Regression, Random Forest, and SVM were compared, allowing different classification approaches to be evaluated.
- **Multiple evaluation metrics:** Accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC were used to provide a comprehensive assessment of model performance.
- **Reproducibility:** Python and Scikit-learn provide a consistent computational environment in which the analysis can be repeated.
- **Application to computational drug discovery:** The project demonstrates how molecular descriptors can be used for preliminary bioactivity prediction and compound prioritization.

10.2 Limitations

Despite these strengths, several limitations should be acknowledged:

- **Synthetic dataset:** The dataset was artificially generated and may not accurately represent experimentally measured bioactivity relationships.

- **Limited features:** Only six molecular descriptors were used. Additional structural descriptors and molecular fingerprints could provide more information.
- **Moderate predictive performance:** The models achieved moderate performance, indicating that the selected features do not completely explain bioactivity.
- **Limited dataset size:** The dataset contained 500 compounds, which may not be sufficient to represent the diversity of real chemical space.
- **Lack of experimental validation:** The predictions were not experimentally verified and therefore cannot be directly considered evidence of biological activity.
- **Generalization limitations:** Performance on the synthetic testing set does not guarantee similar performance on real-world chemical or biological datasets.

10.3 Ethical Considerations

Although this project did not use real patient information, ethical considerations remain important when applying machine learning to healthcare and pharmaceutical research.

First, data privacy and confidentiality must be protected when real healthcare or patient datasets are used. Appropriate anonymization and data-security procedures should be followed.

Second, machine learning predictions should not be treated as definitive medical or biological conclusions without appropriate experimental or clinical validation. Computational predictions should support researchers rather than replace expert judgment.

Third, algorithmic bias and data quality can influence model predictions. Models trained on incomplete or unrepresentative datasets may produce unreliable results when applied to different populations, compounds, or biological systems.

Finally, transparency and reproducibility are essential. The assumptions, dataset-generation methods, preprocessing procedures, model parameters, and limitations should be clearly documented so that computational results can be properly interpreted and independently evaluated.

Overall, responsible application of machine learning requires balancing computational efficiency and predictive capability with data integrity, transparency, privacy, validation, and responsible scientific interpretation.

11. Conclusion

This project successfully demonstrated the application of supervised machine learning techniques to predict the bioactivity classification of chemical compounds using molecular descriptor data. A synthetic healthcare-related dataset was prepared and processed using Python, followed by feature selection, data standardization, model development, hyperparameter tuning, and performance evaluation.

Three classification algorithms—Logistic Regression, Random Forest, and Support Vector Machine (SVM)—were developed and compared. Based on the testing results, Random Forest achieved the highest accuracy (68%), recall (0.700), and F1-score (0.6863) and was therefore selected as the best-performing model according to the F1-score criterion. Logistic Regression achieved the highest ROC-AUC of 0.7456, while SVM produced a closely comparable value of 0.7448.

The results demonstrate that molecular descriptors such as Molecular Weight, LogP, HBA, HBD, TPSA, and Rotatable Bonds can provide useful information for preliminary bioactivity classification. However, the moderate predictive performance and the use of a synthetic dataset highlight the limitations of the present study. Therefore, the findings should be considered a demonstration of a computational machine learning workflow rather than experimentally validated predictions of biological activity.

Overall, the project provided practical experience in Python programming, data preprocessing, machine learning, hyperparameter tuning, model evaluation, scientific visualization, and critical interpretation of computational results. Future work could focus on larger experimentally validated datasets, additional molecular descriptors and molecular fingerprints, improved feature engineering, and more advanced machine learning approaches. Such developments could improve predictive performance and strengthen the application of machine learning in computational drug discovery, medicinal chemistry, and pharmaceutical research.

12. References

- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830.
- Breiman, L. (2001). *Random Forests*. *Machine Learning*, 45, 5–32.
- Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. *Machine Learning*, 20, 273–297.
- Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied Logistic Regression (3rd ed.)*. Wiley.
- Lipinski, C. A., Lombardo, F., Dominy, B. W., & Feeney, P. J. (2001). *Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings*. *Advanced Drug Delivery Reviews*, 46(1–3), 3–26.
- Raschka, S., & Mirjalili, V. (2019). *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2*. Packt Publishing.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter (3rd ed.)*. O'Reilly Media.
- Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. *Computing in Science & Engineering*, 9(3), 90–95.

13. Appendix

The appendix contains the supporting computational materials, Python implementation, model outputs, and detailed evaluation results generated during the predictive modelling study. These materials provide additional evidence for the methodology and results presented in the main report.

13.1 Appendix A: Python Implementation

This appendix presents the Python implementation used for the computational workflow of the project. The code covers synthetic bioactivity dataset generation, data inspection and preprocessing, feature selection, feature standardization, train-test splitting, machine learning model development, hyperparameter tuning using cross-validation, prediction, performance evaluation, confusion matrix generation, ROC-AUC calculation, ROC curve generation, and comparative model analysis.

The implementation uses Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn.

- **Importing Required Libraries:**

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import (
```

```
accuracy_score,  
precision_score,  
recall_score,  
f1_score,  
classification_report,  
confusion_matrix,  
roc_auc_score,  
roc_curve,  
ConfusionMatrixDisplay  
)
```

```
# Reproducibility
```

```
np.random.seed(42)
```

- **Synthetic Bioactivity Dataset Generation:**

```
# Number of synthetic compounds
```

```
n = 500
```

```
# Generate synthetic molecular descriptor dataset
```

```
df = pd.DataFrame({  
    "Compound_ID": [f"CMP{i+1}" for i in range(n)],  
    "Molecular_Weight": np.random.normal(350, 50, n),  
    "LogP": np.random.normal(2.5, 1.0, n),  
    "HBA": np.random.randint(1, 10, n),  
    "HBD": np.random.randint(0, 6, n),
```

```
"TPSA": np.random.normal(70, 20, n),  
"Rotatable_Bonds": np.random.randint(0, 10, n),  
"Bioactivity": np.random.randint(0, 2, n)  
})
```

```
print("Dataset Shape:", df.shape)  
print("\nFirst 5 Rows:")  
print(df.head())
```

- **Data Inspection and Preprocessing:**

```
# Dataset information
```

```
print("Dataset Information:")  
df.info()
```

```
# Missing-value check
```

```
print("\nMissing Values:")  
print(df.isnull().sum())
```

```
# Duplicate-record check
```

```
print("\nDuplicate Rows:", df.duplicated().sum())
```

```
# Statistical summary
```

```
print("\nStatistical Summary:")  
print(df.describe())
```

- **Feature Selection:**

The molecular descriptors used as input features were Molecular Weight, LogP, HBA, HBD, TPSA, and Rotatable Bonds, with Bioactivity used as the binary target variable.

```
# Selected molecular descriptor features
```

```
features = [  
    "Molecular_Weight",  
    "LogP",  
    "HBA",  
    "HBD",  
    "TPSA",  
    "Rotatable_Bonds"  
]
```

```
# Independent variables
```

```
X = df[features]
```

```
# Target variable
```

```
y = df["Bioactivity"]
```

```
print("Selected Features:")
```

```
print(features)
```

- **Training and Testing Data Splitting:**

An 80:20 train-test split was used, resulting in 400 training samples and 100 independent testing samples.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

```
print("Training samples:", X_train.shape[0])  
print("Testing samples:", X_test.shape[0])
```

- **Feature Standardization:**

```
# Initialize StandardScaler
```

```
scaler = StandardScaler()
```

```
# Fit scaler only on training data
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
# Transform testing data using the same scaler
```

```
X_test_scaled = scaler.transform(X_test)
```

```
print("Feature standardization completed.")
```

```
print("Training data shape:", X_train_scaled.shape)
```

```
print("Testing data shape:", X_test_scaled.shape)
```

- **Initial Machine Learning Model Development:**

Three supervised classification algorithms were developed:

1. Logistic Regression
2. Random Forest
3. Support Vector Machine (SVM)

Logistic Regression

```
lr_model = LogisticRegression(  
    random_state=42,  
    max_iter=1000  
)
```

Random Forest

```
rf_model = RandomForestClassifier(  
    random_state=42  
)
```

Support Vector Machine

```
svm_model = SVC(  
    probability=True,  
    random_state=42  
)
```

```
# Model training

lr_model.fit(X_train_scaled, y_train)

rf_model.fit(X_train, y_train)

svm_model.fit(X_train_scaled, y_train)


print("Initial model training completed.")
```

- **Hyperparameter Tuning Using Cross-Validation:**

```
# -----

# Logistic Regression Hyperparameter Tuning
# -----


lr_params = {
    "C": [0.01, 0.1, 1, 10, 100],
    "solver": ["liblinear", "lbfgs"]
}


lr_grid = GridSearchCV(
    estimator=LogisticRegression(
        random_state=42,
        max_iter=1000
    ),
    param_grid=lr_params,
    cv=5,
    scoring="f1"
```

```
)
```

```
lr_grid.fit(X_train_scaled, y_train)
```

```
# -----
```

```
# Random Forest Hyperparameter Tuning
```

```
# -----
```

```
rf_params = {  
    "n_estimators": [50, 100, 200],  
    "max_depth": [None, 5, 10],  
    "min_samples_split": [2, 5]  
}
```

```
rf_grid = GridSearchCV(  
    estimator=RandomForestClassifier(  
        random_state=42  
    ),  
    param_grid=rf_params,  
    cv=5,  
    scoring="f1"  
)
```

```
rf_grid.fit(X_train, y_train)
```

```
# -----  
  
# SVM Hyperparameter Tuning  
  
# -----  
  
svm_params = {  
    "C": [0.1, 1, 10],  
    "kernel": ["linear", "rbf"],  
    "gamma": ["scale", "auto"]  
}  
  
svm_grid = GridSearchCV(  
    estimator=SVC(  
        probability=True,  
        random_state=42  
    ),  
    param_grid=svm_params,  
    cv=5,  
    scoring="f1"  
)  
  
svm_grid.fit(X_train_scaled, y_train)
```

- **Selection of Optimized Models:**

```
# Select the best estimator from each grid search
```

```
best_lr = lr_grid.best_estimator_
```

```
best_rf = rf_grid.best_estimator_
```

```
best_svm = svm_grid.best_estimator_
```

```
# Display optimized parameters
```

```
print("=" * 60)
```

```
print("LOGISTIC REGRESSION")
```

```
print("Best Parameters:", lr_grid.best_params_)
```

```
print("Best CV F1-Score:", round(lr_grid.best_score_, 4))
```

```
print("\nRANDOM FOREST")
```

```
print("Best Parameters:", rf_grid.best_params_)
```

```
print("Best CV F1-Score:", round(rf_grid.best_score_, 4))
```

```
print("\nSUPPORT VECTOR MACHINE")
```

```
print("Best Parameters:", svm_grid.best_params_)
```

```
print("Best CV F1-Score:", round(svm_grid.best_score_, 4))
```

- **Model Prediction:**

```
# Generate predictions on the independent test set
```

```
y_pred_lr = best_lr.predict(X_test_scaled)
```

```
y_pred_rf = best_rf.predict(X_test)
```

```
y_pred_svm = best_svm.predict(X_test_scaled)
```

```
print("Model training and prediction completed.")
```

```
print("Logistic Regression predictions:", len(y_pred_lr))
```

```
print("Random Forest predictions:", len(y_pred_rf))
```

```
print("SVM predictions:", len(y_pred_svm))
```

The expected output should indicate 100 predictions for each model, corresponding to the 100 independent test samples.

- **Detailed Classification Reports:**

```
# Logistic Regression
```

```
print("=" * 60)
```

```
print("LOGISTIC REGRESSION CLASSIFICATION REPORT")
```

```
print("=" * 60)
```

```
print(
    classification_report(
        y_test,
        y_pred_lr,
        target_names=["Inactive", "Active"],
        digits=4
    )
)
```

```
)
```

```
# Random Forest
```

```
print("=" * 60)
```

```
print("RANDOM FOREST CLASSIFICATION REPORT")
```

```
print("=" * 60)
```

```
print(
```

```
    classification_report(
```

```
        y_test,
```

```
        y_pred_rf,
```

```
        target_names=["Inactive", "Active"],
```

```
        digits=4
```

```
    )
```

```
)
```

```
# Support Vector Machine
```

```
print("=" * 60)
```

```
print("SVM CLASSIFICATION REPORT")
```

```
print("=" * 60)
```

```
print(
```

```
    classification_report(
```

```
    y_test,  
    y_pred_svm,  
    target_names=["Inactive", "Active"],  
    digits=4  
)  
)
```

- **Performance Evaluation:**

```
# Probability/decision scores for ROC-AUC calculation
```

```
lr_scores = best_lr.predict_proba(X_test_scaled)[: , 1]
```

```
rf_scores = best_rf.predict_proba(X_test)[: , 1]
```

```
svm_scores = best_svm.decision_function(X_test_scaled)
```

```
# Calculate performance metrics
```

```
results = pd.DataFrame({  
    "Model": [  
        "Logistic Regression",  
        "Random Forest",  
        "SVM"  
    ],
```

```
"Accuracy": [  
    accuracy_score(y_test, y_pred_lr),  
    accuracy_score(y_test, y_pred_rf),  
    accuracy_score(y_test, y_pred_svm)  
],
```

```
"Precision": [  
    precision_score(y_test, y_pred_lr),  
    precision_score(y_test, y_pred_rf),  
    precision_score(y_test, y_pred_svm)  
],
```

```
"Recall": [  
    recall_score(y_test, y_pred_lr),  
    recall_score(y_test, y_pred_rf),  
    recall_score(y_test, y_pred_svm)  
],
```

```
"F1-Score": [  
    f1_score(y_test, y_pred_lr),  
    f1_score(y_test, y_pred_rf),  
    f1_score(y_test, y_pred_svm)  
],
```

```
"ROC-AUC": [  
    roc_auc_score(y_test, lr_scores),  
    roc_auc_score(y_test, rf_scores),  
    roc_auc_score(y_test, svm_scores)  
]  
})  
  
print("\nFinal Model Performance:")  
print(results.round(4).to_string(index=False))
```

- **Confusion Matrix Generation:**

```
# Logistic Regression
```

```
ConfusionMatrixDisplay.from_predictions(  
    y_test,  
    y_pred_lr,  
    display_labels=["Inactive", "Active"]  
)
```

```
plt.title("Logistic Regression Confusion Matrix")  
plt.tight_layout()  
plt.show()
```

```
# Random Forest
```

```
ConfusionMatrixDisplay.from_predictions(
```

```
y_test,  
y_pred_rf,  
display_labels=["Inactive", "Active"]  
)  
  
plt.title("Random Forest Confusion Matrix")  
plt.tight_layout()  
plt.show()
```

```
# Support Vector Machine  
ConfusionMatrixDisplay.from_predictions(  
    y_test,  
    y_pred_svm,  
    display_labels=["Inactive", "Active"]  
)
```

```
plt.title("SVM Confusion Matrix")  
plt.tight_layout()  
plt.show()
```

- **ROC-AUC Calculation and ROC Curve Generation:**

```
# Calculate ROC curves
```

```
fpr_lr, tpr_lr, _ = roc_curve(
```

```
    y_test,  
    lr_scores  
)
```

```
fpr_rf, tpr_rf, _ = roc_curve(  
    y_test,  
    rf_scores  
)
```

```
fpr_svm, tpr_svm, _ = roc_curve(  
    y_test,  
    svm_scores  
)
```

```
# Calculate ROC-AUC values
```

```
auc_lr = roc_auc_score(y_test, lr_scores)  
auc_rf = roc_auc_score(y_test, rf_scores)  
auc_svm = roc_auc_score(y_test, svm_scores)
```

```
# Plot ROC curves
```

```
plt.figure(figsize=(8, 6))
```

```
plt.plot(  
    fpr_lr,  
    tpr_lr,  
    label=f"Logistic Regression (AUC = {auc_lr:.4f})"  
)
```

```
plt.plot(  
    fpr_rf,  
    tpr_rf,  
    label=f"Random Forest (AUC = {auc_rf:.4f})"  
)
```

```
plt.plot(  
    fpr_svm,  
    tpr_svm,  
    label=f"SVM (AUC = {auc_svm:.4f})"  
)
```

Random classifier reference line

```
plt.plot(  
    [0, 1],  
    [0, 1],  
    linestyle="--"  
)
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
```

```
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

- **Final Comparative Model Analysis:**

```
print("=" * 70)
print("FINAL COMPARATIVE MODEL ANALYSIS")
print("=" * 70)
```

```
print(results.round(4).to_string(index=False))
```

```
# Best model according to F1-score
```

```
best_f1_model = results.loc[
    results["F1-Score"].idxmax(),
    "Model"
]
```

```
# Best model according to ROC-AUC
```

```
best_auc_model = results.loc[
```

```
results["ROC-AUC"].idxmax(),  
"Model"  
]  
  
print("\nBest Model based on F1-Score:", best_f1_model)  
print("Best Model based on ROC-AUC:", best_auc_model)
```

13.2 Appendix B: Dataset and Preprocessing Output

This section contains supporting outputs related to the synthetic bioactivity dataset used for model development.

The dataset contained 500 chemical compounds, of which:

- 400 samples were used for training.
- 100 samples were used for independent testing.

The input features consisted of:

- Molecular Weight (MW)
- LogP
- Hydrogen Bond Acceptors (HBA)
- Hydrogen Bond Donors (HBD)
- Topological Polar Surface Area (TPSA)
- Rotatable Bonds (RB)

The target variable was Bioactivity, classified into Active and Inactive categories.

Dataset Shape: (500, 8)

First 5 Rows:

	Compound_ID	Molecular_Weight	LogP	HBA	HBD	TPSA	\
0	CMP1	374.835708	3.426178	7	0	57.447425	
1	CMP2	343.086785	4.409417	1	4	83.967274	
2	CMP3	382.384427	1.101432	8	3	77.334849	
3	CMP4	426.151493	3.062969	3	3	68.125876	
4	CMP5	338.292331	1.849357	6	4	38.531709	

	Rotatable_Bonds	Bioactivity
0	1	1
1	9	1
2	4	1
3	4	1
4	5	0

Generated synthetic bioactivity dataset and first five observations

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Compound_ID                           500 non-null    object
1   Molecular_Weight                       500 non-null    float64
2   LogP                                   500 non-null    float64
3   HBA                                    500 non-null    int64
4   HBD                                    500 non-null    int64
5   TPSA                                  500 non-null    float64
6   Rotatable_Bonds                       500 non-null    int64
7   Bioactivity                           500 non-null    int64
dtypes: float64(3), int64(4), object(1)
memory usage: 31.4+ KB

Missing Values:
Compound_ID      0
Molecular_Weight 0
LogP             0
HBA              0
HBD              0
TPSA             0
Rotatable_Bonds 0
Bioactivity      0
dtype: int64

Duplicate Rows: 0
```

Dataset inspection showing data types, missing values, and duplicate records

Training samples: 400
Testing samples: 100

Training and testing dataset distribution after the 80:20 split

Statistical Summary:

	Molecular_Weight	LogP	HBA	HBD	TPSA
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	350.341900	2.531826	4.964000	2.086000	70.997680
std	49.062662	0.977997	2.558084	1.42642	24.104447
min	187.936633	-0.196887	1.000000	0.000000	4.214203
25%	314.984630	1.904708	3.000000	1.000000	54.379846
50%	350.639857	2.528532	5.000000	2.000000	72.126706
75%	381.839163	3.151242	7.000000	3.000000	86.916389
max	542.636575	5.132382	9.000000	4.000000	136.837774

	Rotatable_Bonds	Bioactivity
count	500.000000	500.000000
mean	5.336000	0.500000
std	3.433173	0.500501
min	0.000000	0.000000
25%	2.000000	0.000000
50%	5.000000	0.500000
75%	8.000000	1.000000
max	11.000000	1.000000

Statistical summary of molecular descriptor features

13.3 Appendix C: Hyperparameter Tuning Output

This section contains the computational output obtained during hyperparameter optimization.

Table: Optimized Hyperparameters and Cross-Validation Performance

Model	Best Hyperparameters	Best CV F1-Score
Logistic Regression	C = 1, solver = liblinear	0.7422
Random Forest	n_estimators = 100, max_depth = 10, min_samples_split = 5	0.7323
Support Vector Machine (SVM)	C = 10, kernel = linear, gamma = scale	0.7294

The corresponding Python console output showing the best parameters and cross-validation F1-scores may be attached below this table as supporting evidence.

13.4 Appendix D: Detailed Classification Reports

This section presents the detailed class-wise performance of the three optimized models on the independent testing dataset.

Table: Logistic Regression Classification Report

=====				
LOGISTIC REGRESSION CLASSIFICATION REPORT				
=====				
	precision	recall	f1-score	support
Inactive	0.64	0.64	0.64	50
Active	0.64	0.64	0.64	50
accuracy			0.64	100
macro avg	0.64	0.64	0.64	100
weighted avg	0.64	0.64	0.64	100

Table: Random Forest Classification Report

```
=====
RANDOM FOREST - CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
Inactive	0.69	0.66	0.67	50
Active	0.67	0.70	0.69	50
accuracy			0.68	100
macro avg	0.68	0.68	0.68	100
weighted avg	0.68	0.68	0.68	100

Table: Support Vector Machine (SVM) Classification Report

```
=====
SVM - CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
Inactive	0.64	0.68	0.66	50
Active	0.66	0.62	0.64	50
accuracy			0.65	100
macro avg	0.65	0.65	0.65	100
weighted avg	0.65	0.65	0.65	100

13.5 Appendix E: Confusion Matrix Outputs

The confusion matrices obtained from the testing dataset were:

Logistic Regression

```
[[ 32  18]
```

```
 [18  32]]
```

Random Forest

```
[[33 17]
 [15 35]]
```

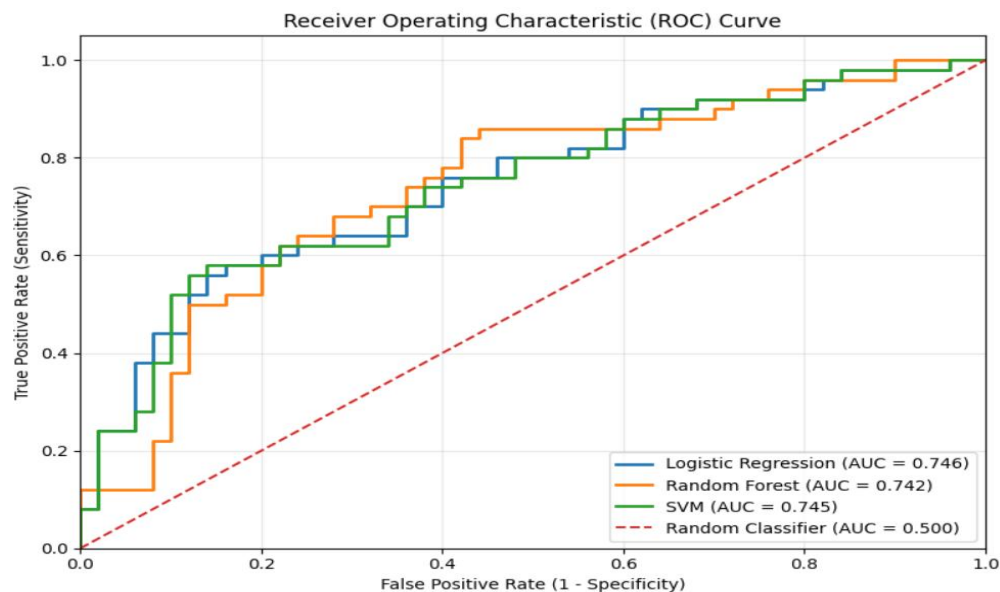
Support Vector Machine (SVM)

```
[[34 16]
 [19 31]]
```

The corresponding confusion matrix figures generated using Python may also be inserted in this section.

13.6 Appendix F: ROC Curve

The ROC curve generated during model evaluation is included in this section to provide a visual comparison of the classification performance of Logistic Regression, Random Forest, and SVM.



Receiver Operating Characteristic (ROC) curves for the three machine learning models

The ROC-AUC values obtained were:

- **Logistic Regression:** 0.7456
- **Random Forest:** 0.7424
- **SVM:** 0.7448

The ROC curve demonstrates the ability of the models to distinguish between active and inactive compounds across different classification thresholds.

13.7 Appendix G: Computational Output and Screenshots

Screenshots of the Python execution environment and important computational outputs may be included in this section as evidence of implementation.

Recommended screenshots include:

Dataset Generation and Initial Inspection:

```
Dataset Shape: (500, 8)
```

```
First 5 Rows:
```

	Compound_ID	Molecular_Weight	LogP	HBA	HBD	TPSA	\
0	CMP1	374.835708	3.426178	7	0	57.447425	
1	CMP2	343.086785	4.409417	1	4	83.967274	
2	CMP3	382.384427	1.101432	8	3	77.334849	
3	CMP4	426.151493	3.062969	3	3	68.125876	
4	CMP5	338.292331	1.849357	6	4	38.531709	

	Rotatable_Bonds	Bioactivity
0	1	1
1	9	1
2	4	1
3	4	1
4	5	0

Generated synthetic bioactivity dataset and initial data inspection

Data Preprocessing:

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Compound_ID           500 non-null    object
1   Molecular_Weight      500 non-null    float64
2   LogP                  500 non-null    float64
3   HBA                   500 non-null    int64
4   HBD                   500 non-null    int64
5   TPSA                  500 non-null    float64
6   Rotatable_Bonds       500 non-null    int64
7   Bioactivity           500 non-null    int64
dtypes: float64(3), int64(4), object(1)
memory usage: 31.4+ KB

Missing Values:
Compound_ID      0
Molecular_Weight 0
LogP             0
HBA              0
HBD              0
TPSA             0
Rotatable_Bonds  0
Bioactivity      0
dtype: int64

Duplicate Rows: 0
```

Dataset structure, missing-value analysis, and duplicate-record check

Statistical Summary:

Statistical Summary:					
	Molecular_Weight	LogP	HBA	HBD	TPSA
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	350.341900	2.531826	4.964000	2.086000	70.997680
std	49.062662	0.977997	2.558084	1.426420	24.104447
min	187.936633	-0.196887	1.000000	0.000000	4.214203
25%	314.984630	1.904708	3.000000	1.000000	54.379846
50%	350.639857	2.528532	5.000000	2.000000	72.126706
75%	381.839163	3.151242	7.000000	3.000000	86.916389
max	542.636575	5.132382	9.000000	4.000000	136.837774

	Rotatable_Bonds	Bioactivity
count	500.000000	500.000000
mean	5.336000	0.500000
std	3.433173	0.500501
min	0.000000	0.000000
25%	2.000000	0.000000
50%	5.000000	0.500000
75%	8.000000	1.000000
max	11.000000	1.000000

Descriptive statistical summary of molecular descriptors

Feature Selection and Train-Test Split:

```
Number of training samples: 400
Number of testing samples: 100
Number of features: 6
```

Feature selection and 80:20 training-testing dataset split

Feature Standardization:

```
Feature standardization completed.
Training data shape: (400, 6)
Testing data shape: (100, 6)
```

Standardization of molecular descriptor features using StandardScaler

Model Training and Prediction:

```
Model training and prediction completed.  
Logistic Regression predictions: 100  
Random Forest predictions: 100  
SVM predictions: 100
```

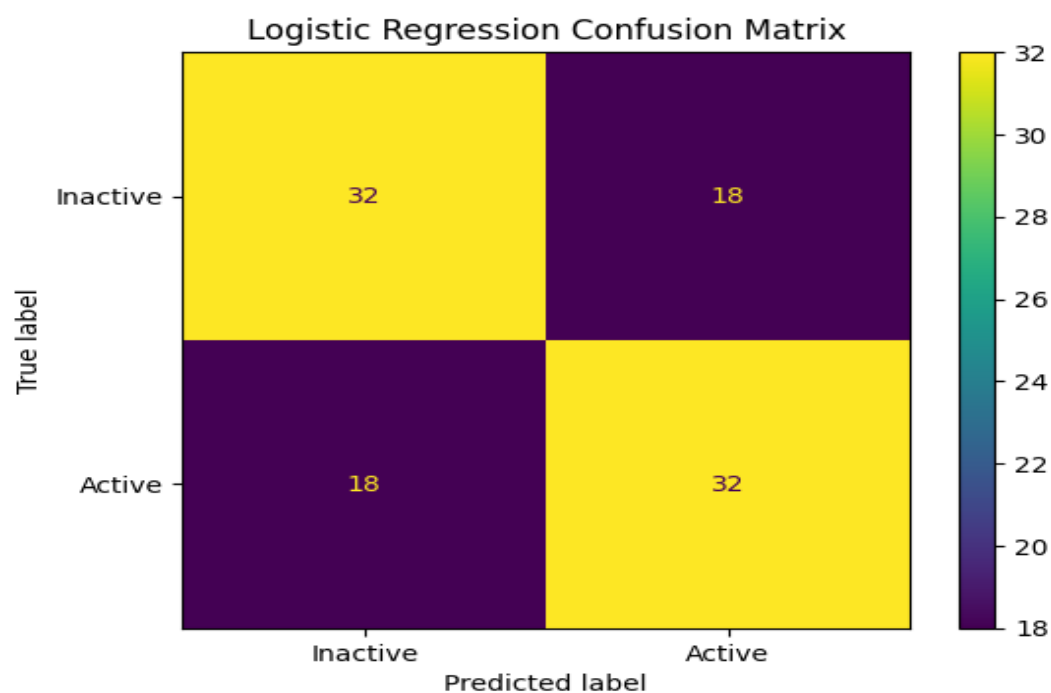
Training and prediction using the optimized machine learning models

Final Model Performance:

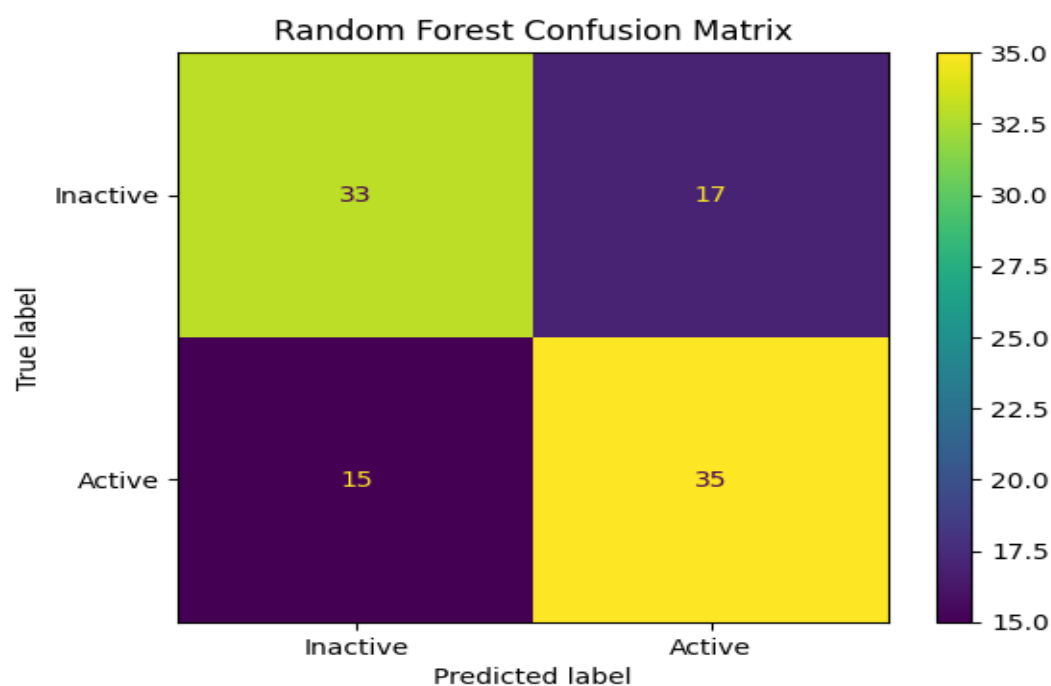
Logistic Regression				
	precision	recall	f1-score	support
Inactive	0.64	0.64	0.64	50
Active	0.64	0.64	0.64	50
accuracy			0.64	100
macro avg	0.64	0.64	0.64	100
weighted avg	0.64	0.64	0.64	100
Random Forest				
	precision	recall	f1-score	support
Inactive	0.69	0.66	0.67	50
Active	0.67	0.70	0.69	50
accuracy			0.68	100
macro avg	0.68	0.68	0.68	100
weighted avg	0.68	0.68	0.68	100
SVM				
	precision	recall	f1-score	support
Inactive	0.64	0.68	0.66	50
Active	0.66	0.62	0.64	50
accuracy			0.65	100
macro avg	0.65	0.65	0.65	100
weighted avg	0.65	0.65	0.65	100
Best Model Based on F1-Score: Random Forest				

Final performance metrics of the optimized machine learning models

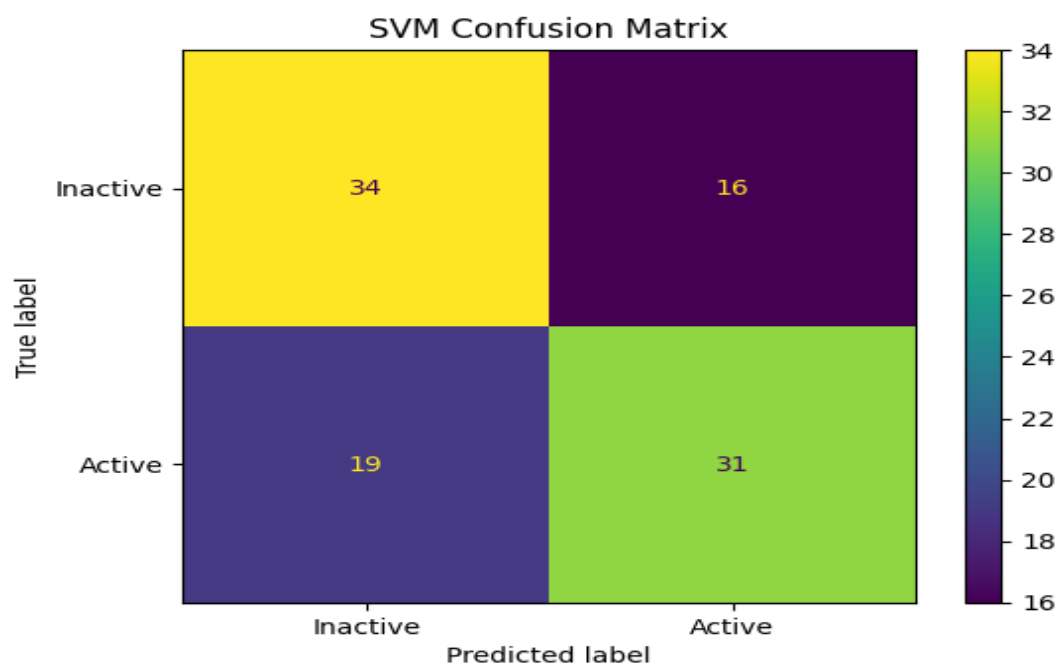
Confusion Matrices:



Confusion matrix of the Logistic Regression model for bioactivity classification

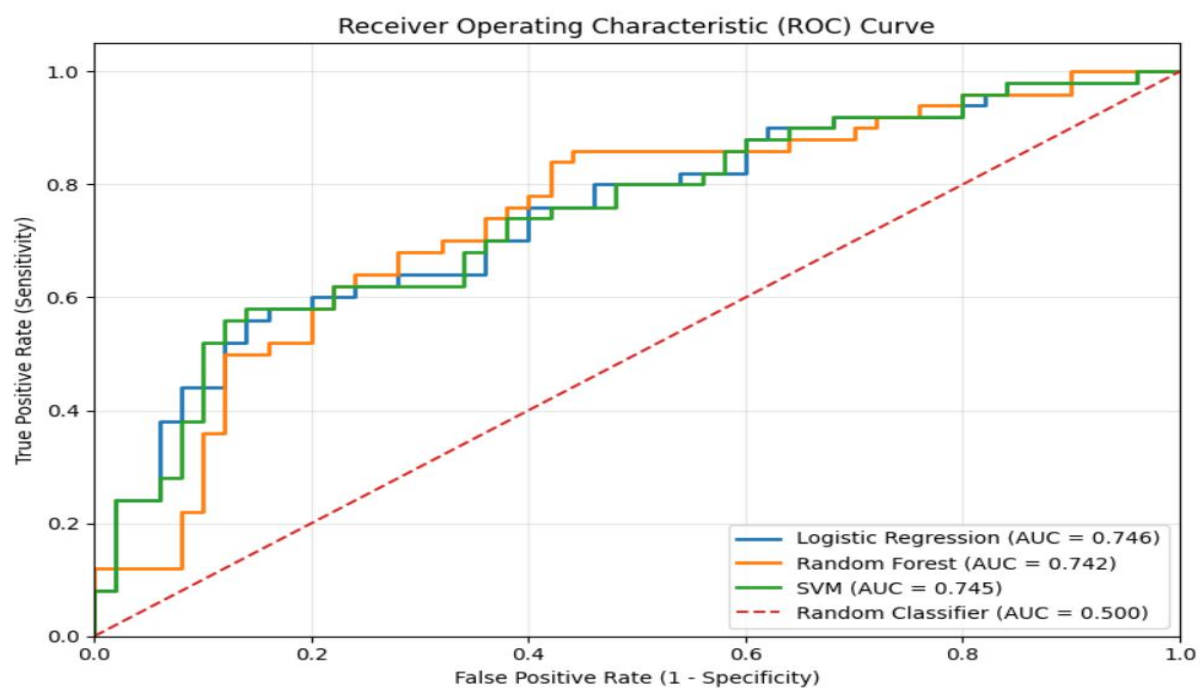


Confusion matrix of the Random Forest model for bioactivity classification



Confusion matrix of the Support Vector Machine (SVM) model for bioactivity classification

ROC Curves:



ROC curve comparison of Logistic Regression, Random Forest, and SVM

Model Comparison Table:

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.64	0.6400	0.64	0.6400	0.7456
1	Random Forest	0.68	0.6731	0.70	0.6863	0.7424
2	SVM	0.65	0.6596	0.62	0.6392	0.7448

Final comparative performance table generated using Python